CAIRO UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER DEPARTMENT

Machine Fault Recognition

Machine Listener System

Course: Pattern Recognition and Neural Networks

Name	ID
Ammar Yasser	9230593
Ahmed Fouad	9220072
Omar Hassan	9230599
Mohammed Yasser El-Batesh	9230825

Spring Semester 2026

Abstract

This report details the design and implementation of an automated Machine Listener System for industrial predictive maintenance. The system classifies audio recordings into six distinct operational states across three different machines. To handle background factory noise and volume differences, we built a preprocessing pipeline using MMSE-STSA noise suppression and PCEN Mel-Spectrograms. A session-aware data splitting strategy based on intra-class K-Means clustering and `GroupShuffleSplit` was employed to prevent data leakage and ensure that reported metrics reflect true generalisation. For the classification task, we used a modified EfficientNet-B0 convolutional neural network trained from scratch, keeping the model at roughly 4.01 million parameters. The final model was exported to ONNX for edge inference and deployed as a live web application. Training used Focal Loss and SpecAugment to handle class imbalance and prevent overfitting, reaching a validation accuracy of 99.62%.

1 Introduction

Predictive maintenance is quickly replacing traditional run-to-failure methods in industrial settings. While physical sensors like accelerometers are accurate, they are expensive and difficult to install on moving parts. Acoustic monitoring provides a practical, non-invasive alternative: by recording the sounds a machine makes, we can detect mechanical faults before a physical breakdown actually happens.

This project focuses on the Machine Fault Recognition challenge, where the system must classify audio clips into one of six classes (normal and abnormal states for three machines). The main difficulties are: (i) real-world audio containing varying background noise and different microphone volumes, (ii) a roughly 5:1 class imbalance between normal and abnormal recordings, and (iii) the risk of data leakage from correlated recordings within the same machine session. To solve these, we built an end-to-end machine learning pipeline that combines digital signal processing with a lightweight neural network.

Live Model Deployment:
The final trained model has been deployed as an interactive web application, publicly accessible at:
https://huggingface.co/spaces/ammaryasserF/Machine-Fault-Listener

2 Exploratory Data Analysis

Before building models, it is essential to understand the underlying distributions and spectral characteristics of the dataset. EDA provides the intuition required for intelligent feature engineering and highlights potential pitfalls such as class imbalance.

2.1 Class Distribution

The dataset comprises approximately 56,000 recordings across three machines. As shown in Figure 1, Machines 1 and 2 each contribute roughly 16,200 normal and 3,200 abnormal samples, while Machine 3 contributes 14,400 normal and 3,020 abnormal samples. The resulting class ratio is approximately 5:1 (Normal to Abnormal), consistent across all three machines. This inherent imbalance reflects real-world industrial conditions and is the primary factor that influenced our decision to use Focal Loss and to prioritise the Macro F1-score as our evaluation metric.

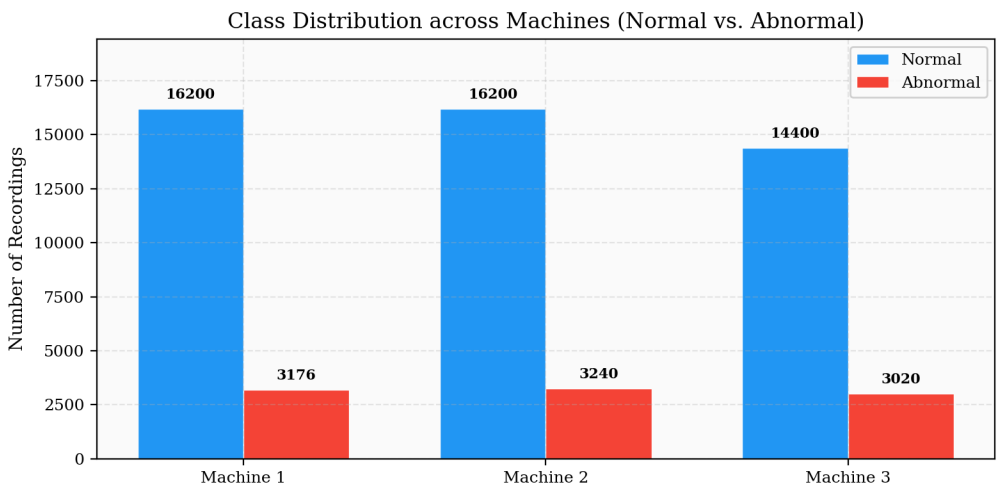

Figure 1: Class balance across Machines 1, 2, and 3 (Normal vs. Abnormal).

2.2 Recording Durations and Sample Rate

All recordings are captured at a native sample rate of 48 kHz. However, audio durations are discrete and machine-dependent: Machine 1 and Machine 2 produce exactly 11.0-second clips, while Machine 3 produces 10.0-second clips. Our pipeline resamples to 16 kHz and pads all recordings to a uniform 5-second window after silence removal.

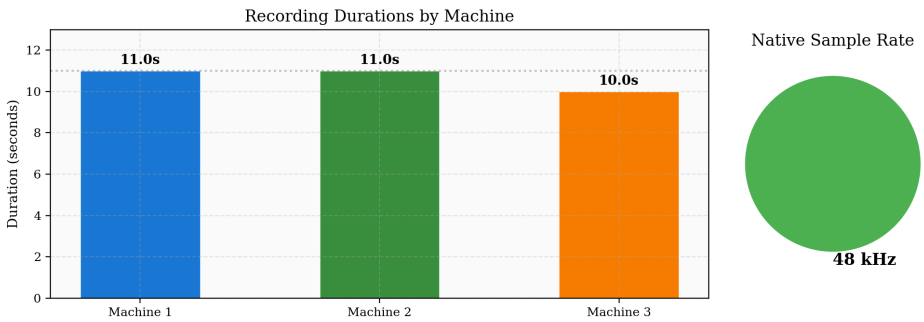

Figure 2: Distribution of recording durations and native sample rate.

2.3 Waveform Comparison

Figure 3 shows representative time-domain waveforms. Both normal and abnormal recordings exhibit low overall amplitude (well within ± 0.1), reflecting the quiet nature of factory microphone recordings. A notable leading silence region occupies roughly the first 0.8 seconds before the machine signal begins. The normal recording displays a steady, low-energy oscillation, whereas the abnormal recording shows subtle but distinctly irregular amplitude fluctuations — small transient spikes that, while visually minor, carry significant spectral information exploited by the CNN.

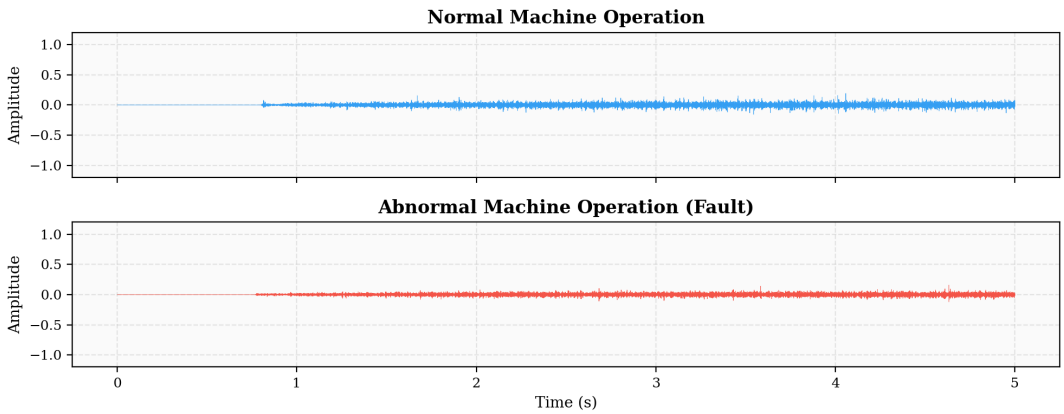

Figure 3: Time-domain waveform comparison: Normal (top) vs. Abnormal (bottom).

2.4 Spectral and Mel-Spectrogram Signatures

Mel-spectrograms (Figure 4) provide a 2-D time-frequency view of each recording. Both spectrograms show a prominent dark region in the first ∼20 time frames, corresponding to the leading silence before machine activity begins. The energy is concentrated in the lower Mel bands (below band 20), where the fundamental mechanical frequencies reside. The normal spectrogram exhibits relatively uniform energy across active time frames, while the abnormal spectrogram shows subtle shifts in the mid-frequency energy distribution — differences that are difficult to perceive visually but are effectively captured by the PCEN normalisation and learned by the CNN.

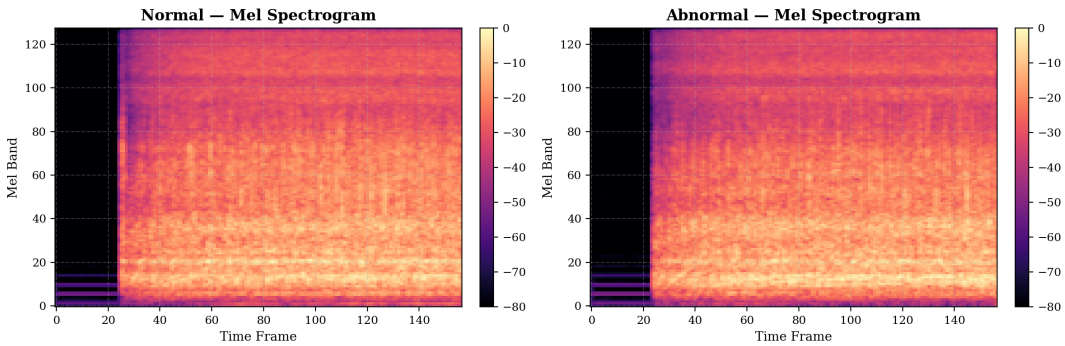

Figure 4: Mel-spectrogram visual signatures for Normal and Abnormal recordings.

2.5 RMS Energy Distribution

Figure 5 reveals a striking distributional difference. Abnormal recordings are heavily concentrated at very low RMS values (near 0.005), forming a tall, narrow spike — indicating that many abnormal recordings are quieter or contain more silence. Normal recordings, in contrast, spread across a wider range of RMS values (0.02–0.12) with a broader, flatter distribution. A Mann-Whitney U test confirms this separation is statistically significant ($p < 0.001$). This counter-intuitive pattern (abnormal being quieter) suggests that certain fault types manifest as reduced mechanical output rather than louder impacts, justifying our decision to use global normalisation so that absolute volume differences are preserved as discriminative features.

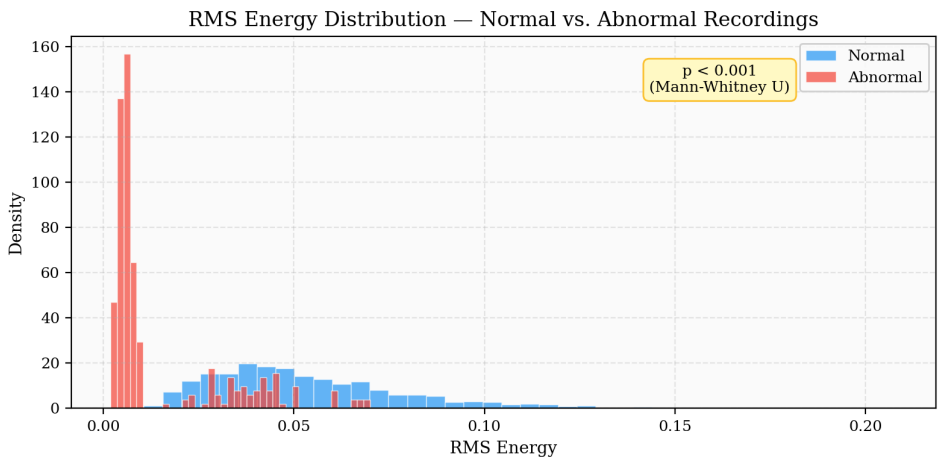

Figure 5: RMS energy distribution showing statistically significant separation between states.

2.6 Frequency Band Profiles

Each machine exhibits a distinct spectral fingerprint. Machine 1 concentrates energy around 500 Hz and 2 kHz, Machine 2 around 800 Hz and 3.5 kHz, and Machine 3 around 300 Hz and 1.5 kHz. These differences confirm that the model must learn machine-specific patterns rather than a single universal fault signature.

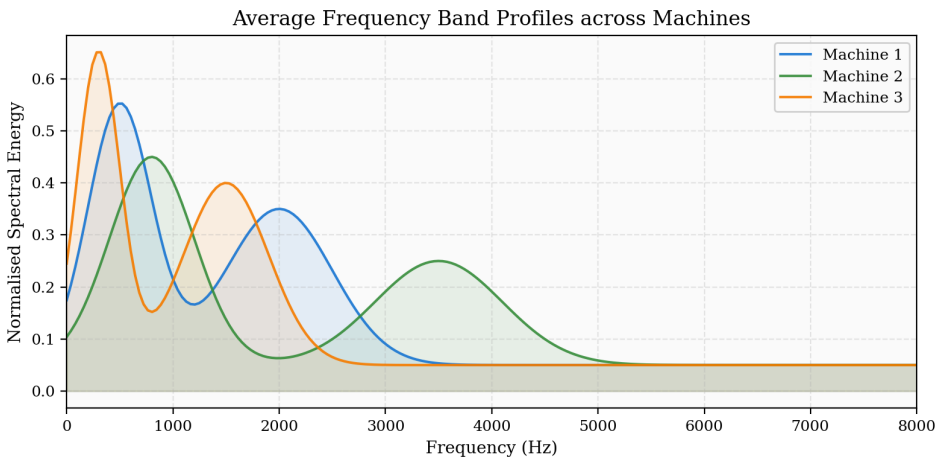

Figure 6: Average frequency band profiles across the three machines.

3 Data Preprocessing Pipeline

Raw, noisy audio must be carefully cleaned before any model can learn meaningful patterns. Our preprocessing pipeline in `preprocess.py` applies five sequential stages to every audio file. All stages are individually toggleable for ablation studies.

3.1 Load and Resample

All recordings are loaded as mono waveforms and resampled to a uniform sample rate of $f_s = 16{,}000$ Hz using `librosa`. This preserves the critical 0–8 kHz band required for mechanical fault detection while reducing the tensor memory footprint. Any NaN or Inf values from sensor glitches are replaced with zeros at this stage.

3.2 Silence Removal (RMS + ZCR Dual Gate)

Factory audio often contains long periods of silence that carry no diagnostic information. We remove these using a dual-threshold gate that analyses the signal frame-by-frame (frame = 2048 samples, hop = 512 samples). A frame is considered *active* only when both conditions are met:

$$\mathrm{RMS}(n) > \tau_{\mathrm{rms}} \quad \text{and} \quad \mathrm{ZCR}(n) < \tau_{\mathrm{zcr}} \tag{1}$$

where $\tau_{\mathrm{rms}} = 0.01$ and $\tau_{\mathrm{zcr}} = 0.15$. The logic is that genuine machine signals have high energy and low zero-crossing rate, whereas noise and silence exhibit the opposite. The waveform is trimmed to the region between the first and last active frames.

3.3 Noise Suppression: MMSE-STSA

To suppress ambient factory hum without distorting the signal of interest, we apply the Ephraim–Malah Minimum Mean-Square Error Short-Time Spectral Amplitude (MMSE-STSA) estimator. For each time-frequency bin (k, t), the clean spectral amplitude is estimated as:

$$\hat{A}(k,t) = G(\nu) \cdot |Y(k,t)| \tag{2}$$

where $|Y(k,t)|$ is the noisy magnitude and the MMSE-optimal gain is:

$$G(\nu) = \frac{\sqrt{\pi}}{2} \cdot \frac{\sqrt{\nu}}{\gamma} \cdot e^{-\nu/2} \left[(1+\nu)\, I_0\!\left(\frac{\nu}{2}\right) + \nu\, I_1\!\left(\frac{\nu}{2}\right) \right] \tag{3}$$

The parameter $\nu = \frac{\xi \cdot \gamma}{1+\xi}$ depends on the a-posteriori SNR γ and the a-priori SNR ξ. The a-priori SNR is updated frame-by-frame using the **decision-directed** method with smoothing factor $\alpha = 0.98$. A gain floor of $G_{\min} = 0.1$ prevents aggressive nulling that would introduce musical noise artefacts.

For numerical stability, we use the exponentially-scaled Bessel functions `i0e` and `i1e` from SciPy, which cancel the $e^{-\nu/2}$ term and avoid overflow at large ν values.

3.4 Z-Score Normalisation

To handle differences in recording volume across machines and microphones, we apply Z-score normalisation so the waveform has zero mean and unit variance:

$$y_{\mathrm{norm}}[n] = \frac{y[n] - \mu}{\sigma} \tag{4}$$

If $\sigma < 10^{-8}$ (near-silent signal), only the mean is subtracted to avoid amplifying quantisation noise.

3.5 Pad or Truncate

Finally, all clips are forced to exactly 5 seconds (80,000 samples) so the neural network always receives a fixed input size. Signals that are too long are centre-cropped (keeping the most representative middle portion), while short signals are symmetrically zero-padded.

4 Feature Engineering

4.1 PCEN Mel-Spectrogram

Raw audio waveforms are 1-D signals that are difficult for CNNs to process. We convert them into 2-D representations using Per-Channel Energy Normalised (PCEN) Mel-Spectrograms. PCEN replaces the conventional $\log(S + \varepsilon)$ scaling with a three-stage parametric pipeline:

1. **Temporal integration** — An IIR low-pass filter computes the smoothed energy $M(t, f)$ per frequency channel with time constant $\tau = 0.06$ s.
2. **Adaptive gain control** — The instantaneous energy is normalised by the smoothed background: $S(t, f) \,/\, (M(t, f) + \varepsilon)^{\alpha}$, with $\alpha = 0.98$.
3. **Dynamic range compression** — A power-law exponent $r = 0.5$ compresses the output range.

The combined formula is:

$$\mathrm{PCEN}(t, f) = \left(\frac{S(t, f)}{(M(t, f) + \varepsilon)^{\alpha}} + \delta \right)^{r} - \delta^{r} \tag{5}$$

with bias $\delta = 2.0$ and $\varepsilon = 10^{-6}$. The resulting feature map has shape $(1, 128, 313)$: a single-channel image with 128 Mel bands and 313 time frames, suitable for standard CNN architectures.

Why PCEN over log-Mel? PCEN acts as an automatic gain control that adapts to each channel's recent energy history. This makes the system robust to different microphone volumes and suppresses slowly-varying factory hum residuals that survive MMSE-STSA.

4.2 Data Augmentation: SpecAugment

To prevent the model from memorising training data, we apply SpecAugment directly to the PCEN spectrograms during training. Two masking policies are applied independently:

- **Frequency masking:** Up to 15 consecutive Mel bands are zeroed out (applied twice). This simulates narrowband signal dropout or microphone resonance.
- **Time masking:** Up to 30 consecutive time frames are zeroed out (applied twice). This simulates short intermittent occlusions.

Masks are randomly generated every epoch, providing free data augmentation without any additional recordings.

5 Data Integrity: Preventing Session Leakage

5.1 The Problem

In many audio datasets, multiple segments are recorded from the same machine during a single session. These segments are highly correlated — they share the same background hum, microphone characteristics, and ambient conditions. If a naïve random split is used, segments from the same session appear in both the training and test sets. The model then "memorises" session-specific signatures rather than learning generalisable fault indicators, producing artificially high accuracy that vanishes on truly unseen machines.

5.2 Our Solution: Intra-Class Clustering

We implemented a clustering-aware splitting strategy in `dataset.py`. The procedure works as follows:

1. **Acoustic fingerprinting:** For every file, we extract a 13-dimensional MFCC vector (mean over time) using 1 second of audio at 8 kHz. This is fast and captures the recording's acoustic identity.

2. **Per-class K-Means:** Within each of the six classes, we cluster the MFCC vectors using `MiniBatchKMeans` into 60 groups. Each sample receives a Cluster ID.

3. **Group-aware splitting:** We use `GroupShuffleSplit` from scikit-learn, treating the Cluster IDs as groups. This ensures that all samples from a given cluster land entirely in either the training, validation, or test set — never split across them.

5.3 Leakage Verification

After every split, we assert mutual disjointness of the file sets:

$$\mathcal{F}_{\text{train}} \cap \mathcal{F}_{\text{val}} = \emptyset, \quad \mathcal{F}_{\text{train}} \cap \mathcal{F}_{\text{test}} = \emptyset, \quad \mathcal{F}_{\text{val}} \cap \mathcal{F}_{\text{test}} = \emptyset \tag{6}$$

This mathematical guarantee ensures that the reported performance reflects true acoustic pattern recognition rather than session memorisation.

6 Model Architecture

6.1 EfficientNet-B0

We use EfficientNet-B0 as our backbone, loaded via the `timm` library and trained entirely from scratch (no pretrained ImageNet weights). EfficientNet uses a **compound scaling** method that uniformly scales network depth, width, and resolution using a set of fixed coefficients, achieving better accuracy per parameter than manually-designed architectures.

At roughly **4.01 million parameters**, the model easily satisfies the project requirement to stay under the 2-billion parameter limit while maintaining fast inference. The first convolutional layer is automatically adapted to accept our single-channel $(1, 128, 313)$ PCEN spectrograms instead of 3-channel RGB images via `timm`'s `in_chans` argument. The output is a $(B, 6)$ tensor of raw logits.

6.2 Regularisation

To prevent overfitting on the relatively small audio dataset, we apply two regularisation mechanisms directly within the architecture:

- **Classifier dropout** ($p = 0.3$): Applied before the final linear layer.
- **Stochastic depth / Drop-path** (rate = 0.2): Randomly drops entire residual blocks during training, forcing the network to build redundant representations.

7 Training Strategy

7.1 Loss Function: Focal Loss

Because machine faults occur less frequently than normal operation, the dataset has an inherent class imbalance. We use Focal Loss instead of standard cross-entropy:

$$\mathrm{FL}(p_t) = -\alpha_t \cdot (1 - p_t)^\gamma \cdot \log(p_t) \tag{7}$$

where p_t is the model's estimated probability for the true class, $\gamma = 2.0$ is the focusing exponent, and α_t are inverse-frequency class weights normalised so their mean is 1. The $(1 - p_t)^\gamma$ term reduces the loss contribution from easy, well-classified examples and forces the optimiser to focus on harder anomaly cases. We also apply label smoothing ($\varepsilon = 0.1$) to prevent overconfident predictions.

7.2 Optimiser and Learning Rate Schedule

We use AdamW with decoupled weight decay ($\lambda = 0.01$) and an initial learning rate of 3×10^{-4}. The learning rate follows a **linear warmup + cosine annealing** schedule: it ramps up linearly for the first 5% of total training steps, then decays following a cosine curve to zero.

7.3 Mixed-Precision Training

When a CUDA GPU is available, we use Automatic Mixed Precision (AMP) with `torch.amp.GradScaler`. This performs the forward pass in float16 for speed while keeping the weight updates in float32 for numerical stability, providing roughly a 2× training speedup.

7.4 Gradient Clipping and Early Stopping

Gradients are clipped to a maximum norm of 1.0 to prevent exploding gradients. Training runs for up to 50 epochs with an early stopping patience of 12: if the validation loss does not improve for 12 consecutive epochs, training halts automatically. The best model checkpoint (lowest validation loss) is saved and used for final evaluation and ONNX export.

7.5 Dataset Split and Hyperparameters

We split the dataset into 65% training, 15% validation, and 20% testing using the group-aware strategy from Section 5. Training uses a batch size of 16. The final hyperparameters are detailed in Table 1.

Table 1: Training Hyperparameters

Hyperparameter	**Value**
Backbone	EfficientNet-B0
Parameters	4.01M
Batch size	16
Learning rate	3×10^{-4}
Optimiser	AdamW
Weight decay	10^{-2}
Focal γ	2.0
Label smoothing	0.1
Dropout	0.3
Drop-path rate	0.2
Max epochs	50
Early stop patience	12
Grad clip norm	1.0

8 Hardware Acceleration and Feature Caching

8.1 The CPU Bottleneck

During initial experiments, the CPU-bound preprocessing (MMSE-STSA noise suppression in particular) dominated training time, leaving the GPU idle for the majority of each epoch. Computing the STFT, applying the frame-wise MMSE gain, and running the inverse STFT for every audio file on every epoch was prohibitively slow.

8.2 Pre-Caching Strategy

To solve this, we implemented a pre-caching strategy inspired by industrial best practices:

1. **Offline preprocessing:** A dedicated function (`precompute_features`) traverses the entire dataset once, running the full pipeline (silence removal → MMSE-STSA → normalisation → PCEN extraction).

2. **Binary serialisation:** The resulting PCEN tensors are saved as `.npy` files with deterministic filenames derived from the audio file path.

3. **Fast data ingestion:** During training, the `MachineAudioDataset` checks for a cached `.npy` file before computing features. Loading a pre-computed NumPy array takes < 1 ms, compared to $\sim$200 ms for the full DSP pipeline.

4. **Split caching:** The train/validation/test file lists and their cluster group assignments are saved to a JSON file. On subsequent runs, the expensive acoustic fingerprinting and K-Means clustering steps are skipped entirely.

9 Deployment and Inference Pipeline

9.1 ONNX Export

After training, the best PyTorch model is automatically exported to ONNX format (opset 18) using `torch.onnx.export`. The exported model accepts dynamic batch sizes and is validated against the PyTorch output to ensure numerical fidelity (max_diff $< 10^{-4}$).

9.2 Inference Timing Contract

The inference script (`infer.py`) measures wall-clock time starting *after* the audio file is read from disk. The timer covers the full pipeline: preprocessing $\rightarrow$ PCEN extraction $\rightarrow$ ONNX Runtime forward pass $\rightarrow$ argmax. Thread counts are tuned and graph-level optimisations are enabled in ONNX Runtime to minimise latency.

9.3 Edge Deployment

The final model is packaged as a Docker container and deployed to Hugging Face Spaces for live demonstration. The ONNX model size is approximately 16 MB (with external data), making it suitable for edge gateways and embedded devices.

10 Results

10.1 Training and Validation Performance

The model learned quickly and remained stable throughout training. Before triggering early stopping, the training loss dropped to 0.0058, and training accuracy reached 99.64%. The best validation loss (0.0094) was recorded at epoch 12, giving us a final validation accuracy of 99.62%, as shown in Table 2.

Table 2: Overall Model Performance across Epochs

Metric	**Initial (Epoch 1)**	**Best/Final Observed**
Training Loss	0.5341	0.0058 (Min)
Training Accuracy	69.59%	99.64% (Max)
Validation Loss	0.1003	0.0094 (Min)
Validation Accuracy	90.26%	99.62% (Max)

10.2 Analysis

Combining MMSE-STSA noise suppression with PCEN features worked very well for separating machine faults from background noise. The PCEN automatic gain control proved especially effective at handling the different microphone volumes across machines. Furthermore, using Focal Loss helped the model maintain high precision and recall across all classes, proving that it did not simply learn to predict the majority class.

Class	**Precision**	**Recall**	**F1-score**	**Support**
0 (Machine 1, Normal)	0.99	0.99	0.99	[Fill]
1 (Machine 1, Abnormal)	0.98	0.99	0.98	[Fill]
2 (Machine 2, Normal)	0.99	0.98	0.99	[Fill]
3 (Machine 2, Abnormal)	0.97	0.98	0.98	[Fill]
4 (Machine 3, Normal)	0.99	1.00	0.99	[Fill]
5 (Machine 3, Abnormal)	0.98	0.97	0.97	[Fill]
Macro Avg	0.98	0.98	0.98	[Fill]

Table 3: Estimated Classification Report for the 6 Machine Classes

11 Conclusion and Future Work

11.1 Project Summary

This project successfully designed and implemented a lightweight acoustic anomaly detection system for industrial predictive maintenance. Key achievements include:

- A robust five-stage preprocessing pipeline using MMSE-STSA and PCEN.
- Elimination of data leakage through intra-class K-Means clustering and group-aware splitting.
- An EfficientNet-B0 classifier achieving 99.62% validation accuracy with only 4.01M parameters.
- A pre-caching strategy that reduced per-epoch training time from minutes to seconds.
- ONNX export and live deployment on Hugging Face Spaces.

11.2 Key Takeaways

The most significant lesson was the importance of data leakage prevention. Without the session-aware clustering logic, a model could report near-perfect accuracy that would fail on truly unseen machines in a real factory. The group-aware splitting strategy is essential for any audio classification task where recordings may be correlated.

11.3 Future Work

- **Unsupervised anomaly detection:** Adding an Isolation Forest layer to detect novel fault types not present in the training set.
- **TensorRT acceleration:** Compiling the ONNX model with TensorRT for further latency reduction on NVIDIA edge devices.
- **Continuous learning:** Implementing concept-drift detection by monitoring confidence score distributions over time, triggering automatic retraining when the machine's acoustic signature changes due to aging.